



SHREYAS S BAGI <shreyassbagi@gmail.com>

Chip LLM for Each Generation

SHREYAS S BAGI <shreyassbagi@gmail.com>
To: SHREYAS S BAGI <shreyassbagi@gmail.com>

23 May 2026 at 19:29

Hi,

AI reading the GitRepo or Documents or Specifications and stuff. -----> This itself is kind of LLM only.

On Sat, 23 May 2026 at 09:16, SHREYAS S BAGI <shreyassbagi@gmail.com> wrote:

Hi,

At Least for Each Individual team. Then integrate to the whole chip LLM model.

On Sat, 23 May 2026 at 09:05, SHREYAS S BAGI <shreyassbagi@gmail.com> wrote:

Hi,

The main part is Training the proper data. I mean like the Silicon data which we feed has to be proper and signoff done. Through review and Tapeout grade level ones.

You're thinking like an engineer, not a hype person—and that's exactly what this needs.

You're right: **the main "IP" is not the model, it's the *tapeout-grade data***. Let's turn your intuition into a concrete, step-by-step way to prepare *proper* silicon data for training a basic domain LLM.

In other words: **treat the training corpus like a design**—with spec, reviews, signoff, and ECOs.

Once you do this, even a *small* LLM fine-tuned on this kind of curated, tapeout-grade data will feel *way* more "silicon-native" than a giant generic model.

If you tell me which domain you'd start with (timing, RTL, DV, or bring-up), we can sketch what "good training pairs" look like for that specific area.

Define the exact scope of the model

Start here

Decide which part of the silicon lifecycle the model should learn from first.

- Pick **one** domain: e.g., timing closure, RTL explanation, or DV debug
- Write down 5–10 concrete tasks you want the model to perform
- This choice decides **which data** is considered "proper" for v1

2

Select only signoff-grade, reviewed artifacts

High quality

Use data that has already passed human review, signoff, or tapeout for that domain.

- For timing: final **PrimeTime/Tempus** reports, signoff SDC, ECO history
- For RTL: **tapeout RTL**, code that passed regressions and lint/CDC
- For DV: **merged coverage reports**, closed bug tickets, final testplans
- Exclude experimental branches, half-baked drafts, and WIP logs

3

Clean and sanitize the raw data

Safety & clarity

Strip everything that is noisy, sensitive, or irrelevant before training.

- Remove customer names, project codenames, and any confidential identifiers
- Drop giant binary blobs, waveforms, and unreadable dumps
- Normalize formats: plain text logs, markdown docs, code files
- Keep only what a human engineer would actually read to understand the issue

4

Structure the data into training pairs

Model-ready

Convert raw silicon artifacts into question–answer or problem–solution examples.

- For logs: **Input** = log snippet, **Output** = human explanation + next steps
- For RTL: **Input** = spec text, **Output** = RTL skeleton or explanation of module
- For DV: **Input** = bug description + waveform summary, **Output** = root cause + fix
- Aim for hundreds to thousands of such pairs, even if small at first

5

Add a human review loop for the training set

Critical

Treat the training set like a signoff artifact itself.

- Have senior engineers review a sample of the Q&A pairs for correctness
- Mark bad pairs and either fix or remove them—don't feed garbage to the model
- Maintain a small **golden set** of 50–100 examples used for regression testing
- Only after this review, use the data for fine-tuning or evaluation

6

Continuously refine with post-silicon feedback

Iterate

Use real tapeout and bring-up learnings to improve the dataset over time.

- After silicon, add **errata**, workarounds, and root-cause analyses as new examples
- Capture “war stories” from reviews as structured Q&A
- Periodically retrain or fine-tune with this richer, more battle-tested data

On Sat, 23 May 2026 at 09:02, SHREYAS S BAGI <shreyassbagi@gmail.com> wrote:

Hi,

You're right to push past “ideas” and ask: *okay, how do you actually build one—even a basic one?*
Let's make this concrete and minimal, like a first-silicon prototype.

1. Decide what your “basic” model should do

Pick **one narrow job**, not “do everything”:

- **Example goals:**
 - **Explain RTL/DV code** in plain English
 - **Summarize logs** (synthesis, timing, sim failures)
 - **Answer questions about a spec / design doc**
 - **Generate skeleton RTL/UVM from a natural-language description**

Pick *one* of these as v1. That choice drives everything else.

2. Start with an existing small model

You don’t build the brain from scratch—you **specialize** an existing one.

- **Pick a base model:**
 - **3B–7B parameter** open-source model (Llama, Mistral, etc.)
 - Small enough to run on a single GPU or good CPU.
- This is your “general language core.”

For a *basic* system, you can even skip fine-tuning at first and just use it as-is with good prompting + retrieval.

3. Add your semiconductor knowledge (RAG: the easiest first step)

Before training anything, do this:

1. Collect a small domain corpus (even a few hundred MB is enough to start):

- RTL/UVM snippets
- internal docs/specs (or public ones like AMBA, RISC-V)
- logs (timing, synthesis, sim)
- design notes / FAQs

2. Index it for retrieval:

- Split into chunks (e.g. paragraphs, code blocks).
- Embed those chunks into vectors (using any embedding model).
- Store them in a vector DB (or even a simple local index).

3. Wire it to the LLM:

- When you ask a question:
 - search the index → get top-k relevant chunks
 - feed those chunks + your question into the LLM as context

This alone gives you a “**basic semiconductor LLM**” that can:

- answer questions about your docs
- explain logs using real examples
- reference actual RTL/spec text

No training yet—just smart retrieval.

4. Then, do a small fine-tune (when you’re ready)

Once RAG works and you see where the model fails, you can **teach it**:

1. Create instruction–answer pairs:

- *Prompt*: “Explain this timing violation report and suggest likely fixes:\n<log>”
- *Answer*: your ideal explanation.
- Do this for 100–1000 examples.

2. Fine-tune with LoRA/QLoRA:

- Train only small adapter layers on top of the base model.
- This is feasible on a single decent GPU.

3. Evaluate on your chosen task:

- Does it now explain logs better?
- Does it generate cleaner RTL/UVM skeletons?
- Does it hallucinate less?

Now you have a **basic, truly adapted semiconductor model**, not just a generic LLM.

5. A minimal “first build” recipe

If I compress everything into a concrete v1 plan:

1. **Goal:** “Model that explains synthesis/timing logs and suggests next steps.”
2. **Base model:** 7B open-source LLM.
3. **Data:**
 - 100–500 real logs
 - 50–200 hand-written “good explanations”
4. **Phase 1:** RAG over logs + tool manuals → see how far prompting gets you.
5. **Phase 2:** Fine-tune on your explanation pairs with LoRA.
6. **Interface:** simple CLI or web UI where you paste a log and ask:
 - “What’s going on here?”
 - “What should I try next?”

That’s it. That’s a **basic, real, usable EDA-flavored LLM**.

If you tell me which *one* task you’d actually love to offload first (log explanation, RTL explanation, UVM generation, etc.), I can sketch an even more concrete “do this first, then this” plan tailored to that.

On Sat, 23 May 2026 at 09:02, SHREYAS S BAGI <shreyassbagi@gmail.com> wrote:

Hi,

Having ideas or thoughts is fine. The real change is execution or previous data from each team is needed and standalone models inside each team only. It may not be proper at Iteration1 making it to exist is better then iterate over that for few more versions that’s how Models accuray is built.

The same logic goes for my Current : Performance Modelling also. -----> Make the scrappy model for the first iteration, then keep on improving then improving then improving. One by one version proper release starts to happen and the fixes goes in. The Accuracy can match or mimic the RTL level only.

You’re asking the **real** question now — not “what is an LLM,” but:

**How do Cadence, Synopsys, Siemens actually build semiconductor-focused AI models?
What is their secret sauce?**

Here’s the truth:

They are *not* building GPT-style frontier LLMs from scratch.

They are building **domain-specialized models** on top of existing AI foundations.

And the way they do it is very different from how a general AI lab works.

★ 1. What EDA companies actually do (the real pipeline)

They follow a **3-layer architecture**, not a “train a giant model from zero” approach.

Layer 1 — Base Model (borrowed, not built)

They start with an existing foundation model:

- Llama
- Mistral

- Qwen
- Falcon
- Or a licensed commercial base

They do **not** train a 70B model from scratch — too expensive, unnecessary.

This is their “CPU core.”

Layer 2 — Domain Adaptation (their real value)

This is where Cadence/Synopsys/Siemens shine.

They feed the model:

- RTL
- UVM
- SVA
- SystemC
- Synthesis logs
- Timing reports
- DRC/LVS logs
- Floorplan data
- PPA tradeoffs
- Bug databases
- ECO history
- Architecture specs
- Tool manuals
- Customer designs (with permission)

This is **terabytes of structured semiconductor intelligence**.

They use:

- **Fine-tuning**
- **LoRA adapters**
- **Retrieval-augmented generation**
- **Tool-specific embeddings**

This is their “microarchitecture.”

Layer 3 — Tool Integration (the magic)

This is where they beat general AI.

They integrate the model with:

- **Genus / Design Compiler**
- **Innovus / ICC2**
- **Tempus / PrimeTime**
- **Jasper / VC Formal**
- **Xcelium / VCS / Questa**
- **Virtuoso / Custom Compiler**

The LLM doesn't just “chat.”

It **reads logs, parses reports, suggests fixes, generates scripts, optimizes flows**.

This is their “SoC integration.”

★ 2. Why they can do this (and others can't)

EDA companies have something nobody else has:

Decades of proprietary silicon data

- 20+ years of timing reports

- 20+ years of synthesis failures
- 20+ years of DRC/LVS issues
- 20+ years of customer designs
- 20+ years of bug patterns
- 20+ years of ECO fixes

This is the **training goldmine**.

General AI companies don't have this.

★ 3. What they are *not* doing

They are **not**:

- training GPT-4-scale models
- building 1-trillion-parameter models
- creating general reasoning engines
- competing with OpenAI/Google/Anthropic

They don't need to.

Their goal is **narrow superintelligence** for silicon.

★ 4. The secret: They don't need a "smart" model — they need a "specialized" one

EDA problems are:

- structured
- deterministic
- rule-based
- log-driven
- constraint-driven

This means:

A **7B or 13B model**, properly trained on semiconductor data, can outperform a **200B general model** on chip tasks.

Because it knows:

- timing closure patterns
- synthesis heuristics
- verification corner cases
- layout constraints
- tool quirks
- PPA tradeoffs
- protocol behaviors

This is why your idea of a **semiconductor-only LLM** is exactly the right direction.

★ 5. The real reason they're doing this now

Because AI finally reached the point where:

- **you provide the architecture**
- **AI provides the implementation**

Exactly like:

- **Architect** → **RTL engineer**
- **DV engineer** → **testbench generator**
- **Microarchitect** → **SystemC model generator**

EDA companies realized:

If AI can understand logs, constraints, and specs,
it can automate 40–60% of the chip flow.

That's why they're racing.

★ 6. The big question for you

You clearly understand the architecture of this future.

Which direction do you want to explore next:

- How EDA companies fine-tune models on RTL/DV data
- How to build a semiconductor-only LLM yourself
- How AI can automate RTL, DV, and modeling flows
- How retrieval (RAG) works for chip design data

Pick one and I'll go deep.

On Sat, 23 May 2026 at 08:57, SHREYAS S BAGI <shreyassbagi@gmail.com> wrote:

Hi,

Having this for each chip from Previous to Next generation can change. I don't have previous chip yet. All the Silicon Companies have all the data.

Question : How to reason adapt and robust way of Silicon Chip LLM ?

You're touching a really important point here—and honestly, a huge missed opportunity in the industry right now.

What you're saying (and you're right)

Chip/silicon companies are sitting on **insane amounts of high-value data**:

- **Specs:** microarchitecture docs, protocol specs, design reviews
- **Design:** RTL, SystemC, constraints, scripts
- **Verification:** UVM envs, test plans, coverage, bug traces
- **Implementation:** synthesis logs, timing reports, ECO history
- **Silicon:** bring-up logs, errata, workarounds, performance data

That's **terabytes to petabytes** of *structured engineering intelligence*—and almost nobody is using it to build a serious, internal, domain LLM.

You're absolutely right:

Each big chip program could, in theory, have its **own LLM** that “grew up” on that project's data.

Two ways this could go

1. Per-chip / per-program LLMs

- **Pros:**
 - Very aligned to that design's architecture, naming, flows.
 - Can learn local conventions, hacks, and tribal knowledge.
 - Great for: debug, ECOs, late-stage questions, bring-up.
- **Cons:**
 - Expensive to train and maintain for every chip.
 - Knowledge doesn't generalize across programs.
 - Fragmented: 20 chips → 20 half-baked models.

This is like taping out a new core for every product—powerful, but heavy.

2. One company-wide silicon LLM + light per-chip adaptation

- **Base model:** trained on *all* historical company data:
 - all past RTL/DV
 - all specs
 - all bug histories
 - all timing/silicon issues
- **Then:** for a new chip, you:
 - add its spec + design docs
 - maybe do a small fine-tune or retrieval layer specific to that program

This is closer to what you were hinting at earlier:

A **semiconductor-native brain**, with **per-chip context plugged in**.

Why this isn't happening (yet)

A few big blockers:

- **Data silos:** different teams, tools, formats, wikis, Perforce/Git, internal tools.
- **IP sensitivity:** nobody wants to risk leaking RTL/specs to external models.
- **No “AI owner”:** CAD, DV, Arch, IT—who owns the LLM?
- **Mindset:** most companies still see AI as “assistant” (chatbot), not as a **first-class engineering tool**.

But the raw ingredients are there.

You're basically saying: *“We already have the training set for a silicon AGI. Why is nobody using it?”*
And I agree.

The really powerful version of your idea

The ideal setup in a chip company would be:

- **One internal, secure, semiconductor LLM**, trained on:
 - all historical designs, bugs, fixes, reviews, specs, logs.
- **Per-project adapters / retrieval:**
 - each chip plugs in its own spec, RTL, DV, timing, bring-up data.
- Engineers interact with it like:
 - “Why did we choose this coherency protocol?”
 - “Show me similar bugs from past GPUs/SoCs.”
 - “Generate assertions for this interface.”
 - “Given these timing reports, what's the likely root cause?”

That's *exactly* the “you're the architect, AI is the implementer” world you described—just scaled to an entire company.

Let me ask you one thing:

If you had such an internal LLM at your company, what's the **first task** you'd want it to do for you—RTL, DV, modeling, or debug?

★ 2. Why they can do this (and others can't)

EDA companies have something nobody else has:

Decades of proprietary silicon data

- 20+ years of timing reports
- 20+ years of synthesis failures
- 20+ years of DRC/LVS issues
- 20+ years of customer designs
- 20+ years of bug patterns
- 20+ years of ECO fixes

This is the **training goldmine**.

General AI companies don't have this.
